

SFML Music Player

By: Muahammad Haseeb

June 24, 2025

Abstract

This document presents a detailed overview of the SFML-based Music Player project, highlighting its design, implementation, and functionality. The report outlines the system architecture, discusses the utilization of core data structures such as queues for playlist management, and illustrates the overall program flow. Furthermore, it explains the integration of SFML for graphical rendering and user interaction, emphasizing how these components collectively enhance usability and performance. The abstract aims to provide readers with a concise yet comprehensive understanding of the project's objectives, methodology, and outcomes.

Contents

1	Project Overview	2
2	Core Features	2
3	Core Concepts & Data Structures	3
3.1	Song Management: Doubly Linked List	3
3.2	Recently Played: Stack	3
3.3	Playlists: Queue	3
4	Program Flow & State Management	3
5	UI Screens Explained	4
5.1	Main Player Screen	4
5.2	Create Playlist Screen	4
5.3	Select Playlist Screen	4
6	File Structure & Persistence	4
7	Flowcharts	6
7.1	Application Logic Flow	6
7.2	Create Playlist Process Flow	7
8	Conclusion	8

1 Project Overview

This project is a graphical music player written in C using the **SFML (Simple and Fast Multimedia Library)** for graphics, audio, and event handling. The application supports playback of **.ogg** music files, library management, playlist creation, and persistent storage. Directory scanning and file management are designed for Windows using **windows.h**.

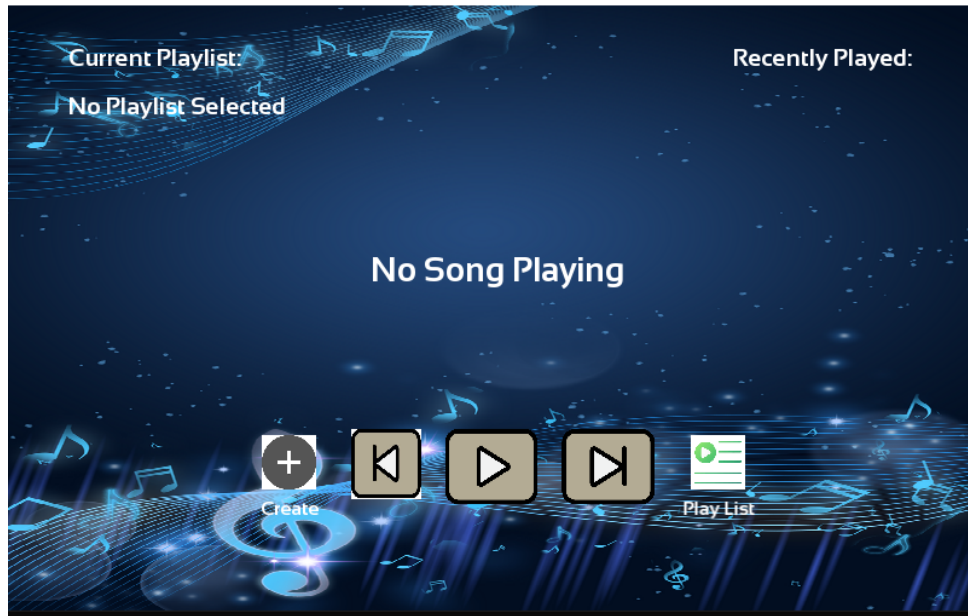


Figure 1: Placeholder for the Main Player UI.

2 Core Features

- **Dynamic Song Loading:** Automatically scans the **music/** directory for **.ogg** files on startup.
 - **Music Controls:** Play, Pause, Next, and Previous.
 - **Playlist Management:**
 - **Create Playlists:** Users can name playlists and select songs.
 - **Play Playlists:** Play any existing playlist.
 - **Persistence:** Playlists saved in **playlists.txt**.
 - **Stateful UI:** Multiple screens handled via a state machine.
 - **User-Friendly Interface:**
 - Displays currently playing song.
 - Shows "Recently Played" songs (up to 5).
 - Displays upcoming songs in the active playlist queue.
-

3 Core Concepts & Data Structures

3.1 Song Management: Doubly Linked List

Structure: struct Song **Purpose:** Store song metadata and enable forward/backward traversal.

```
1 typedef struct Song {
2     char name[100];           // Song title
3     char path[MAX_PATH_LENGTH]; // Full file path
4     struct Song* next;
5     struct Song* prev;
6 } Song;
```

Listing 1: Song Structure

3.2 Recently Played: Stack

Structure: struct StackNode **Purpose:** Maintain LIFO order for the recently played songs (max 5).

```
1 typedef struct StackNode {
2     Song* song;
3     struct StackNode* next;
4 } StackNode;
```

Listing 2: Stack Node for Recently Played

3.3 Playlists: Queue

Structure: struct Playlist and struct PlaylistNode **Purpose:** Store songs in FIFO order to emulate real-world playlists.

```
1 struct Playlist {
2     char name[100];
3     PlaylistNode* front; // Head of the queue
4     PlaylistNode* rear;  // Tail of the queue
5     struct Playlist* next; // Next playlist
6 };
```

Listing 3: Playlist Queue Structure

4 Program Flow & State Management

The application uses a simple enum AppState to manage screens and event logic.

- MAIN_PLAYER: Default music player screen.
 - CREATE_PLAYLIST_SCREEN: User creates a new playlist.
 - SELECT_PLAYLIST_SCREEN: User selects an existing playlist.
-

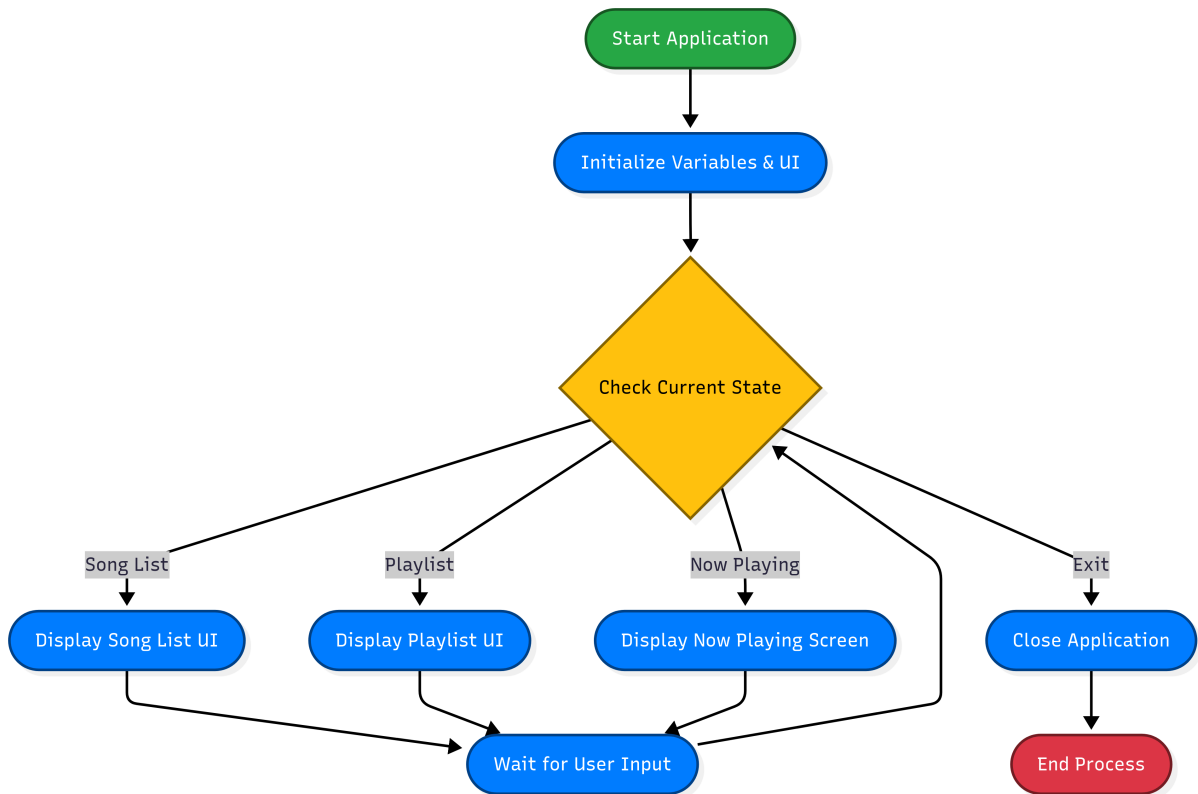


Figure 2: Flowchart: Main loop and state switching.

5 UI Screens Explained

5.1 Main Player Screen

- **Purpose:** Control playback and access playlists.
- **UI Elements:** Playback buttons, currently playing text, "Recently Played" and "Playlist Queue" panels.

5.2 Create Playlist Screen

- **Purpose:** Create and save a new playlist.
- **UI Elements:** Playlist name input, song selection, "CREATE"/"CANCEL" buttons.

5.3 Select Playlist Screen

- **Purpose:** Load and play an existing playlist.
- **UI Elements:** List of playlists, "PLAY SELECTED"/"CANCEL" buttons.

6 File Structure & Persistence

- **music/ Directory:** Contains all .ogg files.

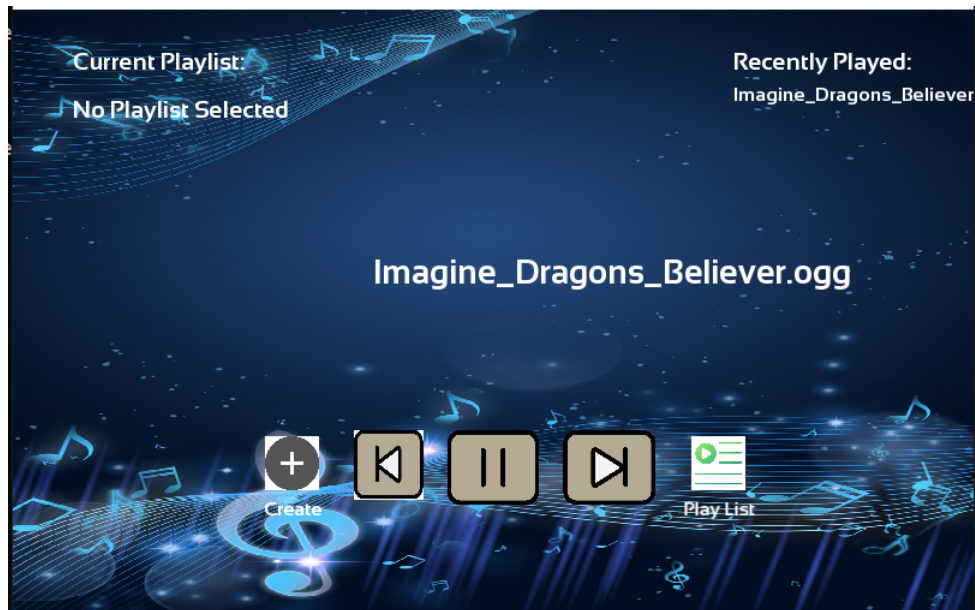


Figure 3: Placeholder for Main Player UI with annotations.

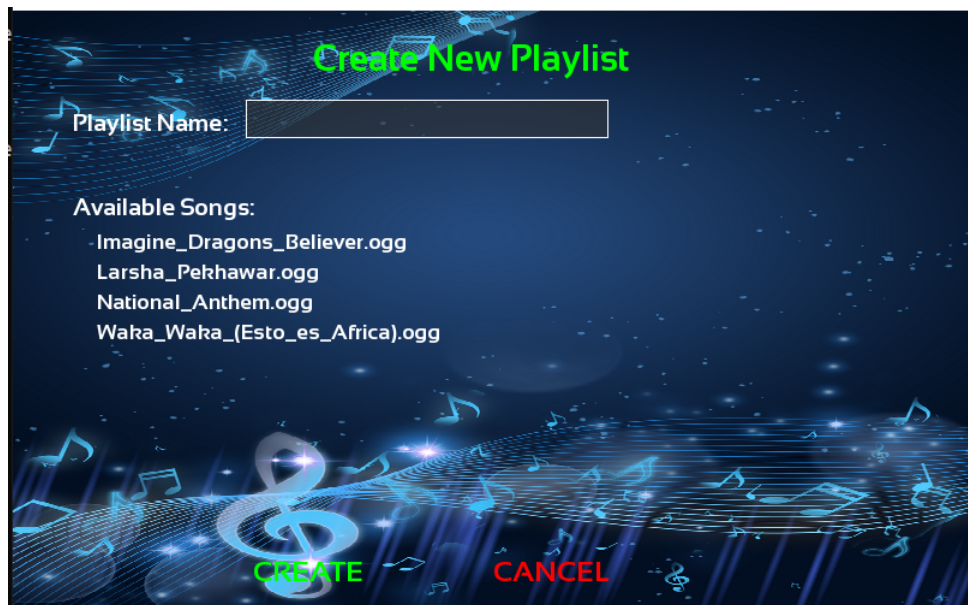


Figure 4: Placeholder for Create Playlist UI.

- **playlists.txt:** Stores user playlists persistently.

```

1 #PLAYLIST_START:Workout Mix
2 C:\path\to\song1.ogg
3 C:\path\to\song3.ogg
4 #PLAYLIST_END
5 #PLAYLIST_START:Chill Vibes
6 C:\path\to\song2.ogg
7 #PLAYLIST_END

```

Listing 4: Example playlists.txt format

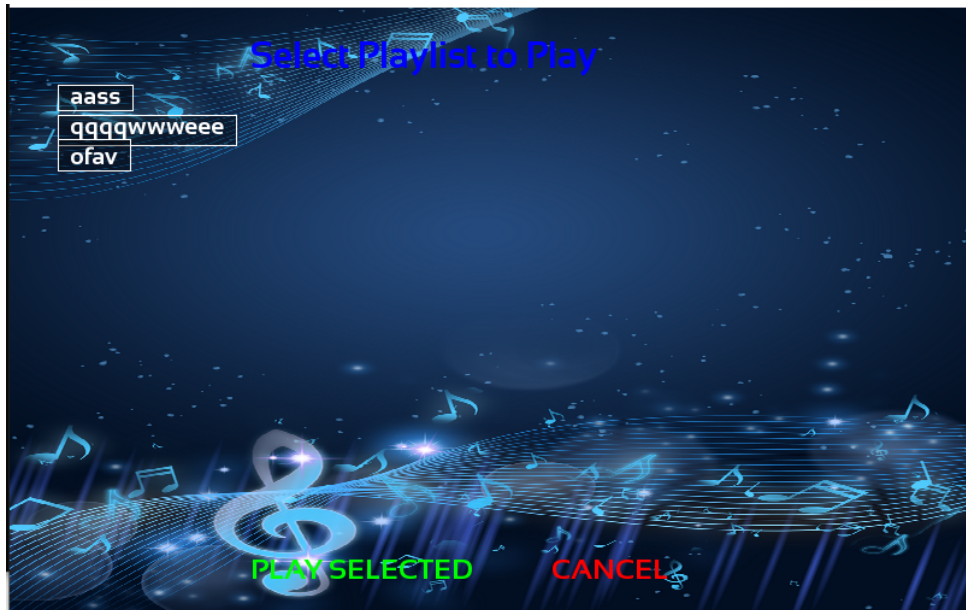


Figure 5: Placeholder for Select Playlist UI.

7 Flowcharts

7.1 Application Logic Flow

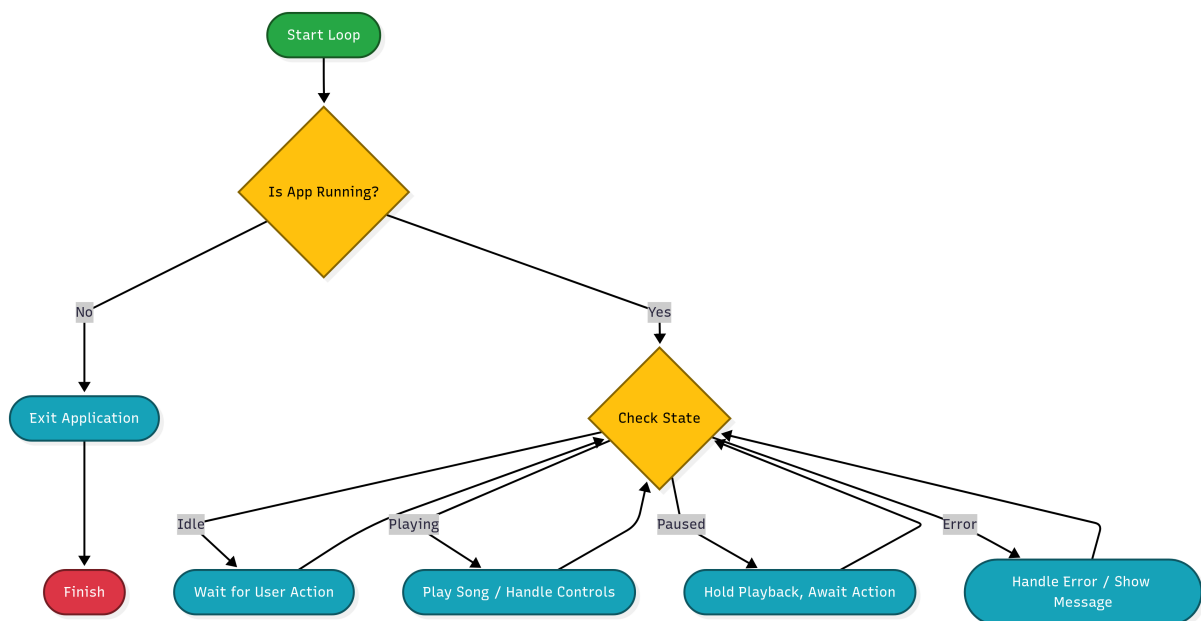


Figure 6: Flowchart: Main loop and state handling.

7.2 Create Playlist Process Flow



Figure 7: Flowchart: Create playlist process.

8 Conclusion

In this project, we successfully implemented a playlist management system with an intuitive flow for adding, organizing, and playing songs. The flowchart illustrates the step-by-step process, highlighting the smooth interaction between user actions and system responses. By scaling the design and structuring the features, the system ensures efficiency, usability, and a clear foundation for future enhancements such as advanced playlist controls and user personalization.